R cheat sheet

This cheat sheet outlines key R commands and options for statistical analysis, as taught in PHCM9795 Foundations of Biostatistics.

Generative AI statement

This work was developed with the assistance of NotebookLM

1. RStudio Setup & Basics

- **Install R:** Download from `https://cran.r-project.org/`. [Section 1.12]
- **Install RStudio**: Download from `https://posit.co/download/rstudio-desktop/`. **You must install R** before installing RStudio. [Section 1.12]
- **RStudio Preferences**: Set to not preserve workspace between sessions by deselecting "Restore .RData into workplace at startup" and choosing "Save workspace to .RData on exit: Never" (Mac: Edit > Settings, Windows: Tools > Global Options). [Section 1.12]
- **Create a Project** to simplify file navigation: **File > New Project... > New Directory > New Project**, then provide a directory name (e.g., PHCM9795) and browse for location. [Section 1.12]
- **Start a new R Script**: **File > New File > R Script** (or **Ctrl+Shift+N/Command+Shift+N**). Write commands here to save your work. [Section 1.12]
- **Run a Script**: **Code > Run Region > Run All** or select all the code, and use **Ctrl+Enter** or **Command+Enter**. [Section 1.12]
- **Comments**: Use `#` to add comments, which R ignores. [Section 1.12]
- **Object Assignment**: Use `<-` (e.g., `x <- 42`) to create objects. [Section 1.12]
- **Vectors**: Best thought of as **columns** of data. Combine values of the same type using the **combine function**: `c()` (e.g., `age <- c(20, 25, 23, 29, 21, 27)`). [Section 1.12]
- **Data Frames**: Essentially a spreadsheet of columns. [Section 1.12]
- **Convert a single column to a data frame:** Some packages (e.g. `jmv`) can only work with data frames, not single columns. Convert a single column to a data frame using `as.data.frame()` (e.g., `age_df <- as.data.frame(age)`). [Section 3.21]
- **Case Sensitivity**: R is case-sensitive (e.g., `age`, `Age`, `AGE` are all different names). [Section 1.12]

2. Packages

- **Install a package**: Use `install.packages("package_name")` (e.g., `install.packages("jmv")`) or **Tools > Install Packages**. Packages **only need to be installed once** for your R installation. [Section 1.12]
- **Load a package**: Use `library(package_name)` (e.g., `library(jmv)`). Must be done in **each R session** before using the package. [Section 1.12]

3. Data Handling & Manipulation

- **Reading** in data:
  - **R data file** (`.rds`): `readRDS("file_path.rds")` (e.g., `pbc <- readRDS("data/activities/mod01_pbc.rds")`). [Section 1.13]
  - **Comma Separated Values** (`.csv`): `read.csv("file_path.csv")` (e.g., `sample <- read.csv("data/examples/mod02_weight_1000.csv")`). [Section 2.20]
  - **Excel file** (`.xlsx`): `read_excel("file_path.xlsx")` from `readxl` package (e.g., `survey <- read_excel("data/examples/mod03_health_survey.xlsx")`). [Section 3.18]
- **View** data:
  - `summary(dataframe)`: Quick overview of all variables. [Section 1.13]
  - `skim(dataframe)` (from `skimr` package): Provides detailed summaries and rudimentary histograms. [Section 1.13]
  - `head(dataframe)`: Shows the first 6 rows. [Section 3.19]
  - `tail(dataframe)`: Shows the last 6 rows. [Section 3.19]
  - `subset(dataframe, condition)`: Lists observations meeting a condition (e.g., `subset(sample, weight>200)`). [Section 1.14]

- **Modify** data:
  - **Create New Variable**: `dataframe$new_variable <- formula` (e.g., `survey$bmi <- survey$weight / (survey$height^2)`). [Section 3.19]
  - **Assign Meaningful Names**: recommended for easy to read output. Simplest way is to create a new column: `dataframe$newvariablename <- datafram$oldvariable`. Spaces and punctuation can be incorporated by surrounding the new variable name in quotation marks: `dataframe$"new variable name (units)" <- datafram$oldvariable`.
  - **Convert to Factor** (Categorical Variable): `dataframe$variable <- factor(dataframe$variable, levels=c(values), labels=c("labels"))` (e.g., `pbc$sex <- factor(pbc$sex, levels = c(1, 2), labels = c("Male", "Female"))`). [Module 2, R notes]
  - **Re-order Factor Levels**: `relevel(dataframe$factor_variable, ref="first_level")` (e.g., `drug$group <- relevel(drug$group, ref="Active")`). Essential for correct calculation of measures of effect like RR or OR. [Section 6.13]
  - **Recode Continuous to Categorical**: `cut(dataframe$continuous_var, breaks=c(limits), labels=c("labels"))` (e.g., `pbc$agegroup <- cut(pbc$age, breaks = c(0, 30, 50, 70, 100), right = FALSE, labels = c("Less than 30", "30 to less than 50", "50 to less than 70", "70 or more"))`). [Section 2.21]
  - **Set value to missing** (NA): `dataframe$variable = ifelse(condition, NA, dataframe$variable)` (e.g., `sample$weight_clean = ifelse(sample$weight==700.2, NA, sample$weight)`). [Section 1.14]
  - **Log transformation**: `dataframe$new_var <- log(dataframe$old_var)` (e.g., `hospital$ln_los <- log(hospital$los+1)`). [Section 9.12]
  - **Back-transform from log**: `exp(log_transformed_value)` (e.g., `exp(3.407232)`). [Section 9.12]

4. Descriptive Statistics (Numerical)

- **Continuous** data:
  - `summary(vector)`: Basic summary (Minimum, 1st Quartile, Median, Mean, 3rd Quartile, Maximum). [Section 1.13]
  - `descriptives(data=dataframe, vars=c(var1, var2), pc=TRUE, ci=TRUE, splitBy=group_var)` (from `jmv`): Comprehensive descriptive statistics, including percentiles, confidence interval of the mean, and can split by a categorical variable. [Section 1.13; Section 3.20]
- **Categorical** data:
  - **One-way Frequency Table**: `descriptives(data=dataframe, vars=variable, freq=TRUE)` (from `jmv`). [Module 2, R notes]
  - **Two-way Frequency Table** (Cross-tabulation): `contTables(data=dataframe, rows=row_var, cols=col_var)` (from `jmv`). Use `pcCol=TRUE` for column percents, and `pcRow=TRUE` for row percents. Use `count=n` for summarised data. [Module 2, R notes; Section 7.11]

5. Descriptive Statistics (Graphical)

- **Continuous** data:
  - **Density plot**: `plot(density(dataframe$variable, na.rm=TRUE), xlab="Label", main="Title")`. Can also use `descriptives()` function in `jmv`, with the `dens=TRUE` option (e.g. `descriptives(data=dataframe, vars=varname, dens=TRUE)`. [Section 1.13]
  - **Box plot**: `boxplot(dataframe$variable, xlab="Label", main="Title")`. Can also use `descriptives()` function in `jmv`, with the `box=TRUE` option. [Section 1.13]
  - **Scatter plot**: `plot(x=dataframe$x_var, y=dataframe$y_var, xlab="X Label", ylab="Y Label")`. Add regression line: `abline(lm(dataframe$y_var ~ dataframe$x_var))`. Can also use `scat(data=dataframe, x="X_var", y="Y_var", line="linear")` from `scatr` package. [Section 8.13]
- **Categorical** data (Bar Charts):
  - **One Categorical Variable**: `plot(dataframe$categorical_var, main="Title", ylab="Number of participants")`. [Section 2.18]
  - **Two Categorical Variables** (from `jmv` package):
    - **Clustered** (side-by-side): `contTables(data=dataframe, rows=row_var, cols=col_var, barplot=TRUE, xaxis="xcols")`. [Section 2.19]
    - **Stacked**: `contTables(data=dataframe, rows=row_var, cols=col_var, barplot=TRUE, xaxis="xcols", bartype="stack")`. [Section 2.19]

  - **Stacked relative frequency**: `contTables(data=dataframe, rows=row_var, cols=col_var, barplot=TRUE, bartype="stack", xaxis="xcols", yaxis="ypc", yaxisPc="column_pc")`. [Section 2.19]
- **Two Categorical Variables** (from `surveymv` package): `surveyPlot(data=dataframe, vars="var_to_plot", group="grouping_var", type="stacked"/"grouped", freq="perc"/"count")`. [Section 2.19]

6. Probability Distributions

While R has functions to calculate probabilities, external applets may be easier.

- **Binomial** probabilities https://homepage.stat.uiowa.edu/~mbognar/applets/bin.html
- **Binomial** distribution in R [Section 2.22]:
  - Probability of exactly x successes: `dbinom(x=k, size=n, prob=p)`.
  - Probability of q or fewer successes: `pbinom(q=k, size=n, prob=p)`.
  - Probability of more than q successes: `pbinom(q=k, size=n, prob=p, lower.tail=FALSE)`.
- **Normal** probabilities https://homepage.stat.uiowa.edu/~mbognar/applets/normal.html
- **Normal** distribution in R [Section 3.22]:
  - Probability of q or less: `pnorm(q, mean, sd)`.
  - Probability of more than q: `pnorm(q, mean, sd, lower.tail=FALSE)`.

7. Confidence Intervals

- **Proportions**: `BinomCI(x=k, n=n, method='wilson')` (from `DescTools` package). [Section 6.11]
- **Mean** (individual data): `descriptives(data=dataframe, vars=variable, ci=TRUE)` (from `jmv`). [Section 3.23]
- **Mean** (summarised data): Use the custom `ci_mean` function provided in the notes, e.g. `ci_mean(n=242, mean=128.4, sd=19.56, width=0.95)`. [Section 3.24]

8. Hypothesis Testing

- **One-Sample T-test**: `t.test(dataframe$variable, mu=hypothesised_mean)`. [Section 4.12]
- **Independent Samples T-test**:
  - `ttestIS(data=dataframe, vars=continuous_var, group=group_var, meanDiff=TRUE, ci=TRUE, welchs=TRUE)` (from `jmv`). [Section 5.9]
  - **Alternative** base R: `t.test(continuous_var ~ group_var, data=dataframe, var.equal=FALSE)` [Section 5.9]
- **Paired T-test:**
  - `ttestPS(data=dataframe, pairs=list(list(i1="var1", i2="var2")), meanDiff=TRUE, ci=TRUE)` (from `jmv`). [Section 5.11]
  - **Alternative** base R: `t.test(dataframe$var1, dataframe$var2, paired=TRUE)`. [Section 5.11]
- **One-Sample Proportion** (Binomial Test): `binom.test(x=successes, n=trials, p=hypothesised_proportion)`. Also `prop.test()` for z-test. [Section 6.12]

Feedback? Contact me at phcm9795.biostatistics@unsw.edu.au